PROJECT OVERVIEW

Proposed Title:

Development of a QR-Based Food and Beverage Ordering and Management System

CHAPTER 1 – INTRODUCTION

1.1 Background

- Growth of digital transformation in the F&B industry
- Increasing adoption of QR-based ordering systems
- Limitations of traditional paper menus and manual ordering

1.2 Problem Statement

- Lack of centralized order management systems
- Inefficient communication between customers, kitchen, and staff
- Difficulty in tracking order status in real-time
- Absence of proper logging and monitoring
- Weak role-based access control in existing solutions

1.3 Objectives

- To develop a QR-based ordering system
- To implement role-based access control (RBAC)
- To enable real-time order tracking
- To integrate online payment (Stripe)
- To provide a logging and monitoring system

1.4 Scope and Limitations

Scope:

- Web-based application
- Multi-tenant system
- QR-based ordering
- Payment integration

Limitations:

- No native mobile application
- No offline functionality
- Payment operates in sandbox/test mode

1.5 Implementation Plan

- Week 1–2: Proposal
- Week 3–6: Analysis & Design
- Week 7–10: Backend Development (Django DRF)
- Week 11–13: Frontend Development (React + TanStack Query)
- Week 14–15: Testing
- Week 16: Final Report

CHAPTER 2 – LITERATURE REVIEW

2.1 Existing Systems

Compare with:

- Food delivery platforms (e.g., GrabFood, ShopeeFood)
- QR-based menu systems
- POS systems

2.2 Technologies

- QR Code technology
- RESTful APIs
- JWT Authentication
- Cloud storage (Cloudinary)
- Payment Gateways (Stripe)

2.3 Related Concepts

- Multi-tenant architecture
- Role-Based Access Control (RBAC)
- Payment gateways (Stripe)

2.4 Gap Analysis

Highlight improvements of my system:

- Custom multi-tenant design
- Full backend control
- Integrated logging system
- Role-specific interfaces

CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

Functional Requirements:

- Scan QR → View menu
- CRUD tables (Tenant Admin)
- CRUD category (Tenant Admin)
- Create order
- Update order status (Chef/Waiter)
- Process payment
- Log system activities (Tenant Admin)
- Create Staff Account (Tenant Admin)

Non-Functional Requirements:

- Performance
- Security (JWT authentication)
- Scalability

3.2 Feasibility Study

- Technical feasibility: Django + React stack
- Economic feasibility: Low development cost
- Operational feasibility: User-friendly
- Time feasibility: 16-week completion

3.3 System Architecture

Include diagram:

- Frontend (React)
- Backend (Django REST Framework)
- Database (PostgreSQL / Supabase)
- Cloudinary (media storage)
- Stripe (payment system)

3.4 Database Design (ERD)

Main entities:

- User
- MenuItem
- Categories
- Tables
- Orders
- OrderItem
- Payments
- Logs

3.5 UML Diagrams

- Use Case Diagram
- Sequence Diagram (Order flow)
- Activity Diagram

3.6 API Design and Documentation (Swagger Integration)

3.6.1 API Overview

- The system uses a **RESTful API architecture**
- All communication between frontend and backend is handled via HTTP requests
- Data is exchanged in **JSON format**

3.6.2 API Structure

My API endpoints list:

Authentication

- `/api/auth/login/`
- `/api/auth/register/`
- `/api/auth/refresh/`

Menu Management

- `/api/menu-items/`
- `/api/menu-items/{id}/`

Orders

- `/api/orders/`
- `/api/orders/{id}/`

Order Items

- `/api/order-items/`
- `/api/order-items/{id}/status/`

Payments

- `/api/payments/`
- `/api/payments/webhook/stripe/`

Logs

- `/api/logs/`
-

3.6.3 Swagger (OpenAPI) Documentation

- The system integrates **Swagger UI** for API documentation
- Swagger provides:
    - Interactive API testing
    - Clear request/response structures
    - Authentication testing (JWT)

3.6.4 Swagger Features in Your System

- Automatically generated API documentation
- Supports testing endpoints directly from browser
- Displays:
    - Request parameters
    - Response schemas
    - Status codes
- Improves developer productivity and debugging

3.6.5 Example API Documentation

3.7 UI/UX Design

- QR menu interface
- Admin dashboard
- Kitchen dashboard
- Waiter interface

CHAPTER 4 – IMPLEMENTATION AND TESTING

4.1 Development

Backend:

- Django + Django REST Framework
- JWT Authentication
- Role-based permissions

Frontend:

- React
- TanStack Query
- Axios

4.2 API Implementation

- Built using **Django REST Framework**
- Endpoints follow REST standards:
    - GET → retrieve
    - POST → create
    - PUT/PATCH → update
    - DELETE → remove

4.3 Swagger Implementation

- Swagger integrated using:
    - drf-yasg / drf-spectacular
- Auto-generates documentation from serializers & views

4.4 Key Feature Implementation

- QR scanning → fetch menu
- Order creation workflow
- OrderItem status updates
- Stripe payment webhook
- Logging system

4.5 API Testing

- APIs tested using:
    - Swagger UI
    - Postman (optional)

Example test cases:

- Login API returns JWT token
- Create order API returns correct data
- Payment webhook processes correctly

4.6 System Guide

- User guide (scan QR, place order)
- Admin guide
- Installation guide

CHAPTER 5 – SUMMARY AND CONCLUSION

5.1 Summary

- Overview of system achievements

5.2 Discussion

- Strengths:
    - Real-time processing
    - Multi-tenant architecture
- Weaknesses:
    - Limited scalability testing

5.3 Limitations

- No offline support
- No mobile application
- Limited UI optimization

5.4 Future Improvements

- Mobile application development
- AI-based recommendations
- Analytics dashboard
- Real-time updates using WebSocket